

Teach Module 23 README: Spring Boot Auto-Configuration

Source Note

The course document was used only to identify the Module 23 topic. The teaching content below is written independently and does not use the course material for instruction.

Module 23 Topic

Spring Boot Auto-Configuration

Key areas:

- Spring Boot value
- Starters
- Embedded server
- Auto-configuration conditions
- Override points
- Application startup lifecycle
- Actuator basics
- Practice lab

Teaching Notes

Spring Boot exists to remove much of the repetitive setup work from Spring applications.

In plain Spring, developers often had to manually configure many infrastructure pieces:

```
DataSource
EntityManagerFactory
DispatcherServlet
Tomcat
Jackson JSON mapper
TransactionManager
ViewResolver
Security filters
```

Spring Boot looks at the libraries on the classpath and the settings in the application, then creates useful default beans automatically.

In simple terms:

```
You add dependencies + write minimal config
Spring Boot creates useful beans automatically
You override only when needed
```

Example: Web Starter

If a project includes:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Spring Boot assumes the application is a web application and automatically configures:

```
Embedded Tomcat
Spring MVC
Jackson JSON serialization
DispatcherServlet
Default error handling
Request mapping infrastructure
```

That is why this minimal application can run as a web API:

```
@SpringBootApplication
public class BankingApplication {
    public static void main(String[] args) {
        SpringApplication.run(BankingApplication.class, args);
    }
}
```

Example controller:

```
@RestController
@RequestMapping("/accounts")
public class AccountController {

    @GetMapping("/{id}")
    public String getAccount(@PathVariable String id) {
        return "Account ID: " + id;
    }
}
```

The developer did not manually create Tomcat, register the dispatcher servlet, or configure JSON support. Spring Boot handled those defaults.

What Is a Starter?

A starter is a curated dependency bundle.

Instead of remembering many related libraries, developers add one starter.

Common starters:

```
spring-boot-starter-web
spring-boot-starter-data-jpa
spring-boot-starter-security
```

```
spring-boot-starter-test
spring-boot-starter-actuator
spring-boot-starter-validation
```

Think of a starter as a sensible toolbox for one kind of job.

Embedded Server

Traditional Java enterprise applications were often packaged as WAR files and deployed to an external server.

Spring Boot commonly uses a different model:

```
The application contains the server.
```

So this command can start the application and server together:

```
java -jar banking-api.jar
```

This model works well for containers, cloud deployment, and local development.

The Main Annotation

Most Spring Boot apps start with:

```
@SpringBootApplication
```

This combines:

```
@Configuration
@EnableAutoConfiguration
@ComponentScan
```

Meaning:

```
@Configuration
This class can define Spring beans.

@EnableAutoConfiguration
Spring Boot should configure beans based on dependencies and properties.

@ComponentScan
Spring should scan this package and subpackages for components.
```

How Auto-Configuration Decides What To Do

Spring Boot uses conditional rules.

Examples:

If Spring MVC is on the classpath, configure Spring MVC.
If Tomcat is on the classpath, start embedded Tomcat.
If Jackson is on the classpath, configure JSON conversion.
If datasource properties exist, configure a database connection.
If the user already defined a bean, do not replace it.

Important principle:

Spring Boot usually backs off when the developer defines their own bean.

Example:

```
@Bean
public ObjectMapper objectMapper() {
    return new ObjectMapper().findAndRegisterModules();
}
```

If the application provides its own `ObjectMapper`, Spring Boot can use that instead of creating a default one.

Common Condition Types

Spring Boot auto-configuration commonly uses conditions like:

```
@ConditionalOnClass
Only configure this if a class exists.

@ConditionalOnMissingBean
Only configure this if the user has not already created a bean.

@ConditionalOnProperty
Only configure this if a property is enabled.

@ConditionalOnWebApplication
Only configure this for a web app.
```

Application Startup Lifecycle

Simplified startup flow:

1. `main()` calls `SpringApplication.run()`
2. Spring determines the application type
3. Environment and properties are loaded
4. `ApplicationContext` is created
5. Components are scanned
6. Auto-configuration classes are evaluated
7. Beans are created and wired
8. Embedded server starts, if this is a web app
9. App becomes ready to receive requests

Configuration Properties

`application.properties` or `application.yml` can influence auto-configuration.

Example:

```
server.port=8081
spring.application.name=banking-api
```

Database example:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/banking
spring.datasource.username=app_user
spring.datasource.password=secret
```

If the correct database and JPA dependencies are present, Spring Boot can use these properties to configure a `DataSource`.

Override Points

Developers commonly customize Spring Boot in three ways:

1. Change properties.

```
server.port=9090
```

2. Define a custom bean.

```
@Bean
public Clock clock() {
    return Clock.systemUTC();
}
```

3. Exclude an auto-configuration.

```
@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)
public class App {
}
```

Use exclusions carefully. Properties or custom beans are often cleaner.

Actuator Basics

Spring Boot Actuator provides operational endpoints for checking and inspecting the app.

Dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Example properties:

```
management.endpoints.web.exposure.include=health,info
```

Endpoint:

```
GET /actuator/health
```

Typical response:

```
{
  "status": "UP"
}
```

Actuator helps answer:

```
Is the app running?
Is the database reachable?
What beans exist?
What config properties are active?
What endpoints are mapped?
```

Mental Model

```
Starter = brings useful dependencies
Auto-configuration = creates useful default beans
Properties = customize defaults
Custom beans = override defaults
Actuator = inspect the running app
```

Practice Exercises

Exercise 1: Create a Minimal Spring Boot Web App

Build a Spring Boot app with:

```
spring-boot-starter-web
```

Create:

```
@RestController
@RequestMapping("/hello")
public class HelloController {

    @GetMapping
    public String hello() {
        return "Spring Boot is running";
    }
}
```

Goal:

Understand that Boot auto-configures Tomcat, Spring MVC, request mapping, and response handling.

Test:

GET http://localhost:8080/hello

Exercise 2: Change the Embedded Server Port

Add:

```
server.port=9090
```

Test:

http://localhost:9090/hello

Goal:

Understand how properties customize auto-configuration.

Exercise 3: Compare Dependencies With and Without a Starter

Run:

```
mvn dependency:tree
```

Look for:

```
spring-web
spring-webmvc
tomcat
jackson
logback
```

Goal:

See how starters pull in useful libraries automatically.

Exercise 4: Add Actuator and Inspect the App

Add Actuator and expose:

```
management.endpoints.web.exposure.include=health,info,beans,mappings
management.info.env.enabled=true
info.app.name=Module 23 Practice App
```

Visit:

```
/actuator/health
/actuator/info
/actuator/beans
/actuator/mappings
```

Goal:

Use Actuator to inspect what Spring Boot configured.

Exercise 5: View the Auto-Configuration Report

Add temporarily:

```
debug=true
```

Restart and inspect the console output.

Look for:

```
Positive matches
Negative matches
Unconditional classes
```

Exercise 6: Override Boot Behavior With a Bean

Create:

```
@Configuration
public class JsonConfig {

    @Bean
    public ObjectMapper objectMapper() {
        ObjectMapper mapper = new ObjectMapper();
        mapper.findAndRegisterModules();
        return mapper;
    }
}
```



```
}  
}
```

Goal:

Understand that Boot backs off when the application provides its own bean.

Exercise 7: Add Validation Starter

Add:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-validation</artifactId>  
</dependency>
```

Create:

```
public class CreateAccountRequest {  
  
    @NotBlank  
    private String ownerName;  
  
    @Positive  
    private BigDecimal openingBalance;  
}
```

Goal:

Observe that validation support appears when the validation starter is present.

Exercise 8: Break Auto-Configuration on Purpose

Temporarily exclude:

```
@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)  
public class DemoApplication {  
}
```

Goal:

Learn how missing properties or excluded auto-configurations affect startup.

Exercise 9: Use Conditional Properties

Create:

```
@Configuration
public class FeatureConfig {

    @Bean
    @ConditionalOnProperty(
        name = "features.audit.enabled",
        havingValue = "true"
    )
    public AuditService auditService() {
        return new AuditService();
    }
}
```

Toggle:

```
features.audit.enabled=true
```

and:

```
features.audit.enabled=false
```

Goal:

Understand condition-based configuration.

Module 23 Lab: Spring Boot Auto-Configuration

Goal

Build a small Spring Boot REST API and observe how Spring Boot configures the application automatically.

Scenario

Create a small **Banking Profile API**. The app should expose REST endpoints, use validation, run on a custom port, and expose Actuator health information.

What You Will Practice

```
Spring Boot starters
Embedded Tomcat
Auto-configuration
application.properties
Validation auto-configuration
Actuator endpoints
Overriding default behavior with your own bean
```

Prerequisites

```
Java 17+
Maven
Spring Boot
IntelliJ or VS Code
Postman, curl, or browser
```

Step 1: Create the Project

Create a Spring Boot Maven project with:

```
Spring Web
Validation
Spring Boot Actuator
```

Dependencies:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Step 2: Configure the App

In `src/main/resources/application.properties`, add:

```
spring.application.name=banking-profile-api
server.port=8085

management.endpoints.web.exposure.include=health,info,mappings
management.info.env.enabled=true
info.app.name=Banking Profile API
info.app.module=Module 23 Spring Boot Auto-Configuration
```

Run the app and visit:

```
http://localhost:8085/actuator/health
```

Expected:

```
{  
  "status": "UP"  
}
```

Step 3: Create a DTO

Create `src/main/java/com/example/banking/dto/CreateProfileRequest.java` :

```
package com.example.banking.dto;  
  
import jakarta.validation.constraints.Email;  
import jakarta.validation.constraints.NotBlank;  
import jakarta.validation.constraints.Size;  
  
public class CreateProfileRequest {  
  
    @NotBlank(message = "Customer name is required")  
    @Size(min = 2, max = 60, message = "Customer name must be between 2 and 60 characters")  
    private String customerName;  
  
    @NotBlank(message = "Email is required")  
    @Email(message = "Email must be valid")  
    private String email;  
  
    public String getCustomerName() {  
        return customerName;  
    }  
  
    public void setCustomerName(String customerName) {  
        this.customerName = customerName;  
    }  
  
    public String getEmail() {  
        return email;  
    }  
  
    public void setEmail(String email) {  
        this.email = email;  
    }  
}
```

Step 4: Create a Response Model

Create `src/main/java/com/example/banking/dto/ProfileResponse.java` :

```
package com.example.banking.dto;  
  
public class ProfileResponse {  
  
    private String id;
```

```

private String customerName;
private String email;
private String status;

public ProfileResponse(String id, String customerName, String email, String status) {
    this.id = id;
    this.customerName = customerName;
    this.email = email;
    this.status = status;
}

public String getId() {
    return id;
}

public String getCustomerName() {
    return customerName;
}

public String getEmail() {
    return email;
}

public String getStatus() {
    return status;
}
}

```

Step 5: Create the Controller

Create `src/main/java/com/example/banking/controller/ProfileController.java` :

```

package com.example.banking.controller;

import com.example.banking.dto.CreateProfileRequest;
import com.example.banking.dto.ProfileResponse;
import jakarta.validation.Valid;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;
import java.util.UUID;

@RestController
@RequestMapping("/profiles")
public class ProfileController {

```

```

@GetMapping
public List<ProfileResponse> getProfiles() {
    return List.of(
        new ProfileResponse("P-1001", "Ava Patel", "ava@example.com", "ACTIVE"),
        new ProfileResponse("P-1002", "Noah Singh", "noah@example.com", "ACTIVE")
    );
}

@GetMapping("/{id}")
public ProfileResponse getProfileById(@PathVariable String id) {
    return new ProfileResponse(id, "Sample Customer", "customer@example.com", "ACTIVE");
}

@PostMapping
public ProfileResponse createProfile(@Valid @RequestBody CreateProfileRequest request) {
    return new ProfileResponse(
        "P-" + UUID.randomUUID().toString().substring(0, 8),
        request.getCustomerName(),
        request.getEmail(),
        "CREATED"
    );
}
}

```

Step 6: Test the Endpoints

Test:

```
GET http://localhost:8085/profiles
```

Expected:

```

[
  {
    "id": "P-1001",
    "customerName": "Ava Patel",
    "email": "ava@example.com",
    "status": "ACTIVE"
  },
  {
    "id": "P-1002",
    "customerName": "Noah Singh",
    "email": "noah@example.com",
    "status": "ACTIVE"
  }
]

```

Test:

```
POST http://localhost:8085/profiles
```

Body:

```
{
  "customerName": "Maya Chen",
  "email": "maya@example.com"
}
```

Expected:

```
{
  "id": "P-xxxxxxx",
  "customerName": "Maya Chen",
  "email": "maya@example.com",
  "status": "CREATED"
}
```

Step 7: Test Validation Auto-Configuration

Send invalid JSON:

```
{
  "customerName": "",
  "email": "not-an-email"
}
```

Expected:

HTTP 400 Bad Request

Reason:

spring-boot-starter-validation enabled Bean Validation support automatically.

Step 8: Inspect Auto-Configured Mappings

Open:

<http://localhost:8085/actuator/mappings>

Find mappings for:

```
/profiles
/profiles/{id}
/actuator/health
/actuator/info
/actuator/mappings
```

This demonstrates that Spring Boot auto-configured:

```
Spring MVC
Controller mappings
Actuator web endpoints
Embedded server routing
```

Step 9: Enable Auto-Configuration Debug Report

Add temporarily:

```
debug=true
```

Restart the app and look for:

```
CONDITIONS EVALUATION REPORT
Positive matches
Negative matches
```

Observe configurations related to:

```
Spring Web
Actuator
Validation
Jackson
Embedded Tomcat
```

Remove `debug=true` after the lab.

Step 10: Override a Default Bean

Create `src/main/java/com/example/banking/config/JacksonConfig.java` :

```
package com.example.banking.config;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class JacksonConfig {

    @Bean
    public ObjectMapper objectMapper() {
        ObjectMapper mapper = new ObjectMapper();
        mapper.enable(SerializationFeature.INDENT_OUTPUT);
        return mapper;
    }
}
```


Restart and call:

```
GET http://localhost:8085/profiles
```

Expected:

JSON response should be pretty-printed.

Practice goal:

Understand that when you define your own bean, Spring Boot can use it instead of its default.

Lab Questions

1. What did `spring-boot-starter-web` auto-configure?
2. Why did the app start on port `8085` ?
3. What happened when you added `spring-boot-starter-actuator` ?
4. Why did validation work without manually creating a validator?
5. What does `/actuator/mappings` show?
6. What is the purpose of `@SpringBootApplication` ?
7. What does `debug=true` reveal?
8. How did the custom `ObjectMapper` change application behavior?

Success Criteria

You are done when:

The app runs on port 8085
GET /profiles returns JSON
POST /profiles accepts valid input
POST /profiles rejects invalid input
/actuator/health returns UP
/actuator/mappings shows your REST endpoints
You can explain which parts were auto-configured by Spring Boot